

REPORTE FINAL

Calculadora de Grafos “con Listas de Adyacencia”

Estructura de Datos

Docente: Miguel Vargas Lomeli

Tecnologías: Java, Java Swing, Apache NetBeans

Integrantes:

- Manuel Silva Madrid
- Erika Audrey Cárcamo Munguía

Repositorio del proyecto:

github.com/Manuul-88/proyecto-grafo

INTRODUCCIÓN

Con base en estos conceptos, se desarrolló una calculadora de grafos utilizando Java y Java Swing, implementada mediante listas de adyacencia y listas enlazadas. La aplicación permite crear nodos, conectar vértices mediante aristas con peso y trabajar con grafos dirigidos y no dirigidos.

Además de la parte lógica, el sistema incorpora una interfaz gráfica interactiva donde es posible visualizar el comportamiento del grafo y ejecutar algoritmos como DFS, BFS, Dijkstra y Kruskal. También se añadieron herramientas visuales como zoom, resaltado de rutas y temas personalizados para mejorar la interacción con el usuario.

OBJETIVOS

El propósito principal del proyecto fue desarrollar un sistema gráfico capaz de representar y manipular grafos mediante listas de adyacencia y listas enlazadas. Para ello fue necesario integrar tanto la parte lógica de la estructura de datos como una interfaz visual que permitiera interactuar directamente con el grafo generado.

Además de la representación gráfica, también se buscó implementar algoritmos clásicos relacionados con teoría de grafos, permitiendo analizar caminos, recorridos y conexiones entre vértices dentro de la misma aplicación.

Entre los objetivos específicos del proyecto se encuentran:

- Implementar un grafo utilizando listas de adyacencia.
- Utilizar listas enlazadas para almacenar conexiones dinámicas.
- Implementar recorridos DFS y BFS.
- Implementar el algoritmo de Dijkstra.
- Implementar el algoritmo de Kruskal.
- Permitir grafos dirigidos y no dirigidos.
- Diseñar una interfaz gráfica interactiva.

DESCRIPCIÓN GENERAL DEL PROYECTO

La aplicación desarrollada funciona como una calculadora de grafos capaz de crear estructuras dinámicas mediante nodos y aristas conectadas entre sí. El usuario puede interactuar directamente con el sistema desde una interfaz gráfica donde es posible insertar vértices, establecer conexiones, asignar pesos y ejecutar algoritmos para analizar recorridos y caminos mínimos dentro del grafo.

Internamente, el proyecto fue implementado mediante listas de adyacencia debido a que este modelo resulta más eficiente para representar grafos dinámicos. Cada nodo almacena una lista enlazada con sus conexiones, permitiendo agregar y eliminar relaciones sin depender de tamaños fijos de memoria.

La estructura del proyecto fue dividida en varias clases para mantener una mejor organización del código y separar responsabilidades dentro del sistema. La lógica principal del grafo se encuentra en la clase `TGrafo`, mientras que las clases `TLista` y `TNodo` administran las listas enlazadas utilizadas para almacenar las conexiones.

Por otra parte, la representación gráfica y la interfaz visual fueron implementadas mediante las clases `PanelGrafo` y `VentanaPrincipal`, encargadas de dibujar los nodos, aristas y controles interactivos del sistema. Esto permitió mantener separada la lógica interna de la representación visual, facilitando el mantenimiento y expansión del proyecto.

IMPLEMENTACIÓN DE LA ESTRUCTURA DE DATOS

La estructura principal utilizada en el proyecto fue la lista de adyacencia, ya que permite representar grafos de forma eficiente almacenando únicamente las conexiones existentes entre vértices. A diferencia de una matriz de adyacencia, este modelo evita desperdicio de memoria cuando el número de conexiones es variable.

Cada vértice contiene una lista enlazada con todos sus nodos adyacentes. A su vez, cada conexión almacena información relacionada con el destino y el peso de la arista, permitiendo trabajar con grafos ponderados y ejecutar algoritmos relacionados con rutas mínimas.

Dentro del sistema se implementaron operaciones como:

- Inserción y eliminación de nodos.
- Inserción y eliminación de aristas.
- Modificación de pesos.
- Verificación de adyacencia.
- Recorridos sobre el grafo.

FUNCIONES IMPLEMENTADAS

Entre las funciones principales implementadas se encuentran:

- Agregar nodos.
- Eliminar nodos.
- Agregar aristas con peso.
- Eliminar aristas.
- Editar pesos.
- Verificar adyacencia.
- Mostrar lista de adyacencia.
- Contar conexiones.
- Limpiar el grafo completo.

ALGORITMOS IMPLEMENTADOS

El proyecto incluye los siguientes algoritmos:

- Recorrido en profundidad (DFS/BPF).
- Recorrido en amplitud (BFS).
- Algoritmo de Dijkstra.
- Algoritmo de Kruskal.

El recorrido DFS fue implementado utilizando recursividad para explorar primero los caminos más profundos antes de retroceder. Por otro lado, BFS utiliza una cola para recorrer primero los vértices más cercanos al nodo inicial.

En el caso de Dijkstra, el algoritmo permite calcular rutas mínimas dentro de grafos dirigidos utilizando pesos positivos. Finalmente, Kruskal fue implementado para generar árboles de expansión mínima dentro de grafos no dirigidos, seleccionando las aristas de menor costo sin formar ciclos.

La integración de estos algoritmos permitió complementar la parte estructural del proyecto con herramientas de análisis sobre el comportamiento del grafo.

INTERFAZ GRÁFICA

Uno de los aspectos principales del proyecto fue el desarrollo de una interfaz gráfica interactiva utilizando Java Swing y Graphics2D. La aplicación fue diseñada para que el usuario pudiera manipular el grafo de forma visual, observando directamente los cambios realizados sobre la estructura.

El sistema gráfico incluye herramientas como:

- Zoom dinámico.
- Minimapa.
- Movimiento de nodos.
- Resaltado de rutas.
- Curvas Bézier.
- Física de movimiento.
- Temas visuales personalizados.

Figura 1. Interfaz principal del sistema.

Se observa la ventana principal de la calculadora de grafos, donde se encuentran los controles para nodos, aristas, recorridos, algoritmos y configuración visual con distintos TEMAS.

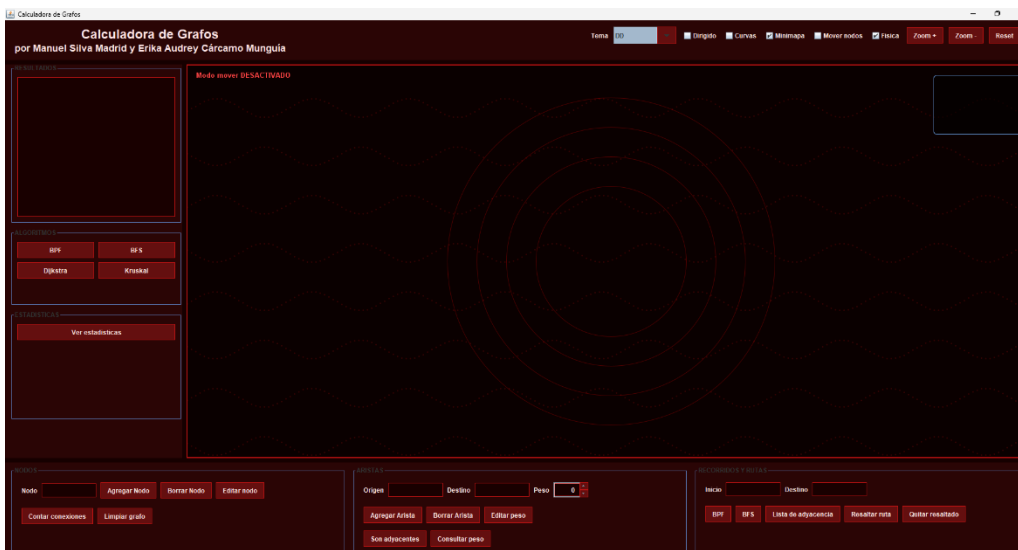
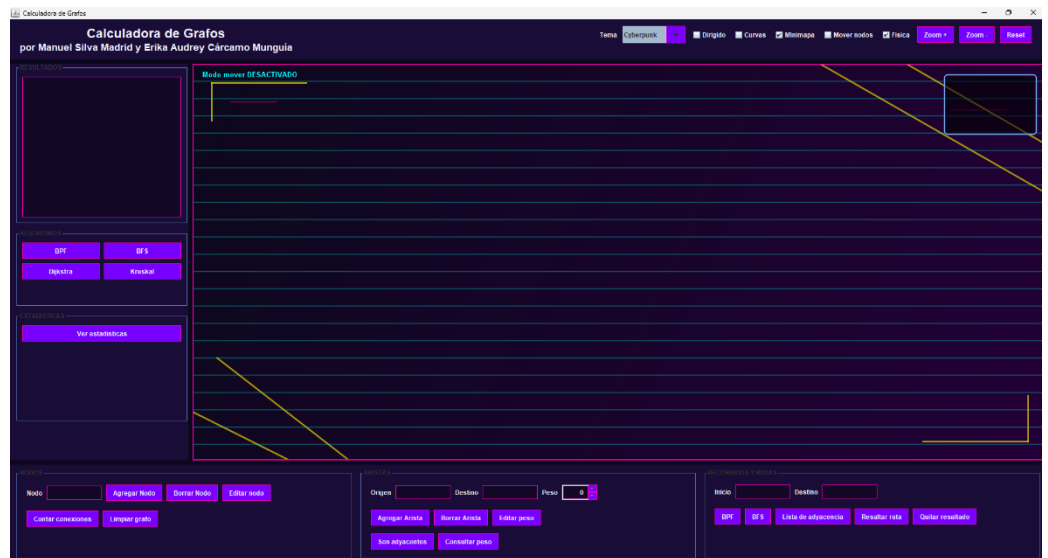
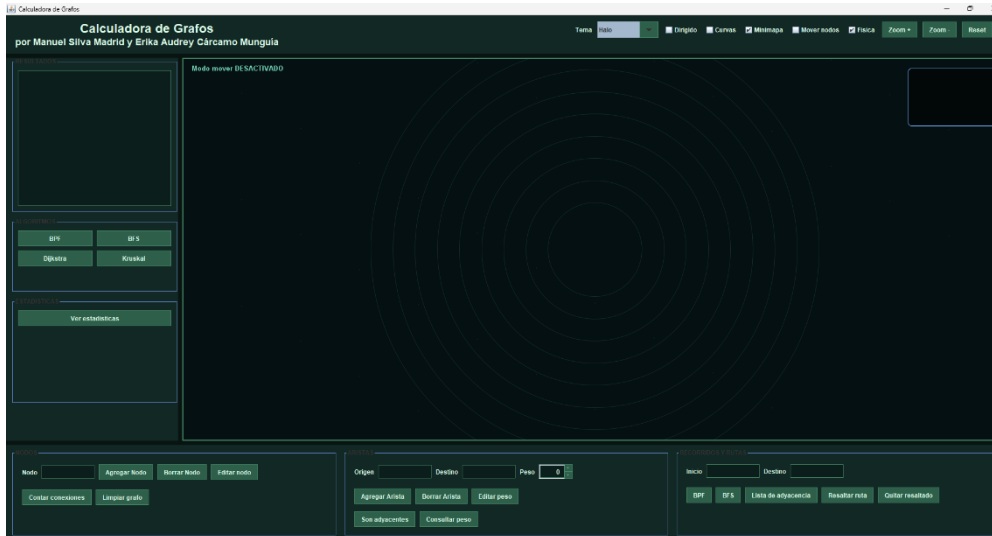


Figura 2. Creación de nodos y aristas con peso.

Se muestra la inserción de vértices y conexiones ponderadas dentro del grafo.

NODOS

Nodo

ARISTAS

Origen Destino Peso

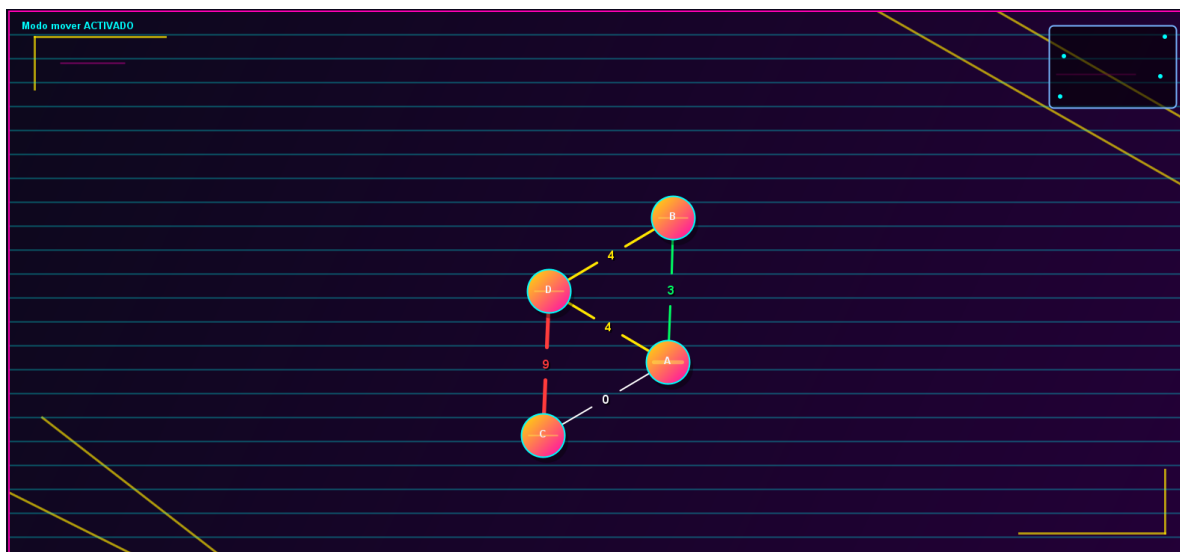


Figura 3. Lista de adyacencia generada.

La captura evidencia la representación interna del grafo mediante listas de adyacencia.

RECORRIDOS Y RUTAS

Inicio Destino

RESULTADOS

Modo NO DIRIGIDO

Lista de adyacencia

```
A -> B(3) -> C(0) -> D(4)
B -> A(3) -> D(4)
C -> A(0) -> D(9)
D -> A(4) -> C(9) -> B(4)
```

Figura 4. Recorrido en profundidad BPF/DFS.

Se muestra el resultado del recorrido desde un nodo inicial.

RECORRIDOS Y RUTAS

Inicio

RESULTADOS

Recorrido BPF

D A B C

Figura 5. Recorrido en amplitud BFS.

Se observa el orden de visita generado mediante cola.



Figura 6. Ejecución del algoritmo de Dijkstra.

La captura muestra los caminos mínimos y costos obtenidos desde el nodo origen.

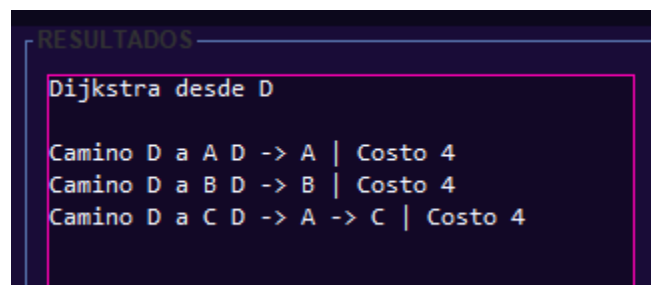
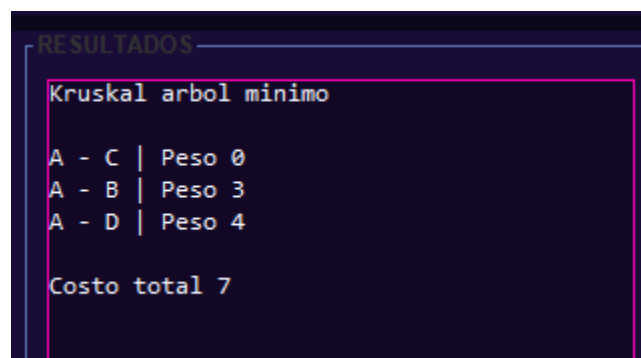


Figura 7. Ejecución del algoritmo de Kruskal.

Se muestra el árbol de expansión mínima generado en un grafo no dirigido.



CONCLUSIONES

El desarrollo de este proyecto permitió aplicar de manera práctica los conocimientos relacionados con estructuras de datos vistos durante el curso, especialmente aquellos relacionados con listas enlazadas, listas de adyacencia y teoría de grafos.

La implementación de una calculadora de grafos permitió comprender mejor el funcionamiento de distintos algoritmos de recorrido y análisis de caminos, además de reforzar conceptos relacionados con programación orientada a objetos y diseño modular de aplicaciones.

Por otra parte, el uso de Java Swing permitió desarrollar una interfaz gráfica interactiva capaz de representar visualmente el comportamiento del grafo, facilitando la comprensión de los recorridos y conexiones entre vértices.

Finalmente, el proyecto ayudó a fortalecer habilidades relacionadas con organización de código, diseño de interfaces, implementación de algoritmos y resolución de problemas, integrando distintos temas de la materia dentro de una sola aplicación funcional y visualmente interactiva.

REFERENCIAS

- Weiss, Mark Allen. *Estructuras de Datos y Algoritmos*. Pearson Educación.
- Cormen, Thomas H.; Leiserson, Charles E.; Rivest, Ronald; Stein, Clifford. *Introduction to Algorithms*. MIT Press.
- Goodrich, Michael T.; Tamassia, Roberto. *Data Structures and Algorithms in Java*. Wiley.
- Material y apuntes proporcionados durante el curso de Estructura de Datos.
- Repositorio oficial del proyecto:
<https://github.com/Manuul-88/proyecto-grafo>